

M08 - Files, APIs & Data Ingestion

17-lesson learner book | Beginner-first professional institute standard | B01-style teaching contract

Purpose

Ingest files, APIs and databases with reliable state, retries, quarantine and replay safety.

Course boundary

This is the independent Standalone Data Engineer course, not Four-Course C3. Complete the course in sequence; each new idea is still taught from first principles.

Option B institute identity | Charcoal #1F2937 | Teal #14B8A6 | Soft Blue #3B82F6

This book is your teacher

Use one lesson at a time. Do not read protected review before saving a genuine attempt. Every lesson follows the same beginner-first sequence even when the engineering idea becomes more technical.

Daily lesson loop

1) why it matters -> 2) what you learn -> 3) prerequisite -> 4) picture it -> 5) new words -> 6) useful visual/source -> 7) follow once -> 8) expected result -> 9) why it worked -> 10) mistakes -> 11) recovery -> 12) independent task -> 13) validate -> 14) teach it back -> 15) protected review + changed retry -> 16) evidence/mastery + exact next route.

Prerequisite check

Complete M05 Python-for-DE and M07 PostgreSQL/data-modeling foundations. You should already preserve raw inputs and validate keys/counts.

Attempt protection

A protected review is repair material, not a shortcut. Save the first attempt, open only the matching review if needed, record the repair, close review, then solve a changed retry from a clean state.

Contents / Index

Every formal lesson and its actual page number in this exact rebuilt book.

DE08-01 - Ingestion architecture: source -> landing -> staging	5
DE08-02 - Defensive CSV ingestion	7
DE08-03 - JSON and nested-record ingestion	9
DE08-04 - Parquet ingestion and metadata inspection	11
DE08-05 - REST API contracts, status codes and timeouts	14
DE08-06 - API authentication, pagination and rate limits	16
DE08-07 - Retries, backoff, jitter and transient failures	18
DE08-08 - Database extraction and safe query boundaries	20
DE08-09 - Historical batch landing and raw metadata	22
DE08-10 - Incremental loads and high-water marks	24
DE08-11 - CDC concepts and change-event semantics	26
DE08-12 - Schema drift, compatibility, time zones and quarantine	28
DE08-13 - Checkpoint/state management and replay safety	30
DE08-14 - Idempotent landing and content identity	32
DE08-15 - Late-arriving data, event time and processing time	34
DE08-16 - Ingestion observability: counts, latency and run metadata	36
DE08-17 - Ingestion integration, reconciliation and backfill proof	38

Prerequisite, tool and outcome map

ASSUMED

Complete M05 Python-for-DE and M07 PostgreSQL/data-modeling foundations. You should already preserve raw inputs and validate keys/counts.

MODULE PURPOSE

Ingest files, APIs and databases with reliable state, retries, quarantine and replay safety.

EXIT EVIDENCE

Module practical + fresh retest + saved evidence; no critical unresolved failure.

Protected study route

Teaching -> assigned visual/source when useful -> follow once -> close worked material -> independent attempt -> validate -> save evidence -> teach it back. If weak: matching protected review -> targeted repair -> close review -> changed fresh retry -> revalidate.

DE08-01 - Ingestion architecture: source -> landing -> staging

1. Why it matters

Design a reproducible source-to-landing-to-staging boundary and state what is immutable, validated and transformed at each layer. This matters because ingestion is where outside uncertainty enters the system; weak boundaries can lose, duplicate or reshape data before anyone notices.

2. What you are learning

Design a reproducible source-to-landing-to-staging boundary and state what is immutable, validated and transformed at each layer.

3. Prerequisite / what you already know

Complete M05 Python-for-DE and M07 PostgreSQL/data-modeling foundations. You should already preserve raw inputs and validate keys/counts.

4. Picture it in your head

Picture three zones: source outside your control, landing as an immutable receipt area, and staging as the workspace where parsing/validation begins. Do not blur those responsibilities.

5. New words before we use them

Ingestion is a chain of responsibility. Source is what the producer exposes. Landing preserves raw evidence as received. Staging parses and validates it into a shape safe for later transformation. Do not overwrite raw bytes just because a later schema is easier to query. A trustworthy pipeline can show what arrived, what was rejected, and what changed afterward.

6. Useful video / source

Source: DataTalksClub — Docker/PostgreSQL data-engineering workshop

Assigned scope: Use matching data-ingestion / script / database chapter

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=IP8xXebHmuE)

URL: <https://www.youtube.com/watch?v=IP8xXebHmuE>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Create folders source/, landing/, staging/, quarantine/. Copy one batch file into landing without renaming its bytes; record size + SHA-256;

parse it into staging only after validation. Open the matching Guided Labs section. Reproduce the pattern once with the normal ParcelFlow fixtures while saving commands/output rather than transcribing the lesson.

8. Expected result / what you should see

You should be able to design a reproducible source-to-landing-to-staging boundary and state what is immutable, validated and transformed at each layer and show a concrete result/state that another person can inspect. Observable target: For Ingestion architecture: source -> landing -> staging: record the actual before/after state and verify the lesson requirement with an independent control.

9. Why it worked

The workflow is trustworthy because the received input is identified and preserved before transformation, so every downstream result can be traced back to unchanged source evidence. Your independent check is: Do not validate Ingestion architecture: source -> landing -> staging by appearance alone. The proof is complete, replayable receipt/state evidence with no silent loss or duplication.

10. Common beginner mistakes

1. Calling Ingestion architecture: source -> landing -> staging complete because the command or tool finished. 2. Skipping the independent control: Do not validate Ingestion architecture: source -> landing -> staging by appearance alone. 3. Changing several things after a failure instead of isolating the first broken step. 4. Overwriting the failed attempt instead of saving it as repair evidence.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Picture three zones: source outside your control, landing as an immutable receipt area, and staging as the workspace where parsing/validation begins. 3. Run the earliest independent control: Do not validate Ingestion architecture: source -> landing -> staging by appearance alone. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Design a 4-layer folder/contract for a weather-station file feed. State exactly what can change at each boundary and what evidence must never be overwritten.

13. Validate your result

Do not validate Ingestion architecture: source -> landing -> staging by appearance alone. Use the lesson-specific control: Save the source/input identity, intended row/event grain, counts before/after, explicit error/quarantine evidence where relevant, and one independent reconciliation. Record the actual observed value/state and compare it with what the task requires. If they differ, the lesson is not yet mastered.

14. Teach it back in your own words

Without notes, explain Ingestion architecture: source -> landing -> staging in your own words. Start with the purpose: Design a reproducible source-to-landing-to-staging boundary and state what is immutable, validated and transformed at each layer. Then name one realistic failure mode and explain how you would prove the result using this check: Do not validate Ingestion architecture: source -> landing -> staging by appearance alone.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Ingestion architecture: source -> landing -> staging with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE08-02.

DE08-02 - Defensive CSV ingestion

1. Why it matters

Read CSV defensively with explicit encoding/header/delimiter/schema expectations, quarantine malformed business rows and reconcile counts. This matters because ingestion is where outside uncertainty enters the system; weak boundaries can lose, duplicate or reshape data before anyone notices.

2. What you are learning

Read CSV defensively with explicit encoding/header/delimiter/schema expectations, quarantine malformed business rows and reconcile counts.

3. Prerequisite / what you already know

DE08-01 mastered.

4. Picture it in your head

Defensive CSV ingestion assumes delimiters, encodings, columns and bad rows may surprise you. Parse into a controlled schema and quarantine violations instead of silently coercing them.

5. New words before we use them

CSV is deceptively simple: delimiters, quotes, newlines, encodings, headers, missing fields and business-invalid values can all break assumptions. Defensive ingestion separates parsing validity from business validity, counts every source business row, and quarantines bad rows with a reason instead of silently dropping them.

6. Useful video / source

Source: DataTalksClub — Docker/PostgreSQL data-engineering workshop

Assigned scope: Use matching data-ingestion / script / database chapter

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=IP8xXebHmuE)

URL: <https://www.youtube.com/watch?v=IP8xXebHmuE>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Use `csv.DictReader`. Require the expected header, parse `ship_date` as a real date and `weight_kg` as numeric. Send bad rows to quarantine and prove seen = accepted + quarantined. Open the matching Guided Labs section. Reproduce the pattern once with the normal `ParcelFlow` fixtures while saving commands/output rather than transcribing the lesson.

8. Expected result / what you should see

You should be able to read CSV defensively with explicit encoding/header/delimiter/schema expectations, quarantine malformed business rows and reconcile counts and show a concrete result/state that another person can inspect.

Observable target: For Defensive CSV ingestion: Run the independent input from a clean state, record the actual output/count/schema/hash or exception, and compare it with an independently stated expected control.

9. Why it worked

This works because read CSV defensively with explicit encoding/header/delimiter/schema expectations, quarantine malformed business rows and reconcile counts is made explicit before the result is accepted. Your independent check is: Do not validate Defensive CSV ingestion by appearance alone. The proof is complete, replayable receipt/state evidence with no silent loss or duplication.

10. Common beginner mistakes

1. Calling Defensive CSV ingestion complete because the command or tool finished. 2. Skipping the independent control: Do not validate Defensive CSV ingestion by appearance alone. 3. Changing several things after a failure instead of isolating the first broken step. 4. Overwriting the failed attempt instead of saving it as repair evidence.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Defensive CSV ingestion assumes delimiters, encodings, columns and bad rows may surprise you. 3. Run the earliest independent control: Do not validate Defensive CSV ingestion by appearance alone. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Write a CSV reader for a changed shipment file with one invalid date and one missing required ID. Quarantine errors with reason and prove count reconciliation.

13. Validate your result

Do not validate Defensive CSV ingestion by appearance alone. Use the lesson-specific control: Save the source/input identity, intended row/event grain, counts before/after, explicit error/quarantine evidence where relevant, and one independent reconciliation. Record the actual observed value/state and compare it with what the task requires. If they differ, the lesson is not yet mastered.

14. Teach it back in your own words

Without notes, explain Defensive CSV ingestion in your own words. Start with the purpose: Read CSV defensively with explicit encoding/header/delimiter/schema expectations, quarantine malformed business rows and reconcile counts. Then name one realistic failure mode and explain how you would prove the result using this check: Do not validate Defensive CSV ingestion by appearance alone.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Defensive CSV ingestion with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE08-03.

DE08-03 - JSON and nested-record ingestion

1. Why it matters

Parse nested JSON without silently dropping arrays/objects, flatten only the fields required by the target grain and preserve raw payload evidence. This matters because ingestion is where outside uncertainty enters the system; weak boundaries can lose, duplicate or reshape data before anyone notices.

2. What you are learning

Parse nested JSON without silently dropping arrays/objects, flatten only the fields required by the target grain and preserve raw payload evidence.

3. Prerequisite / what you already know

DE08-02 mastered.

4. Picture it in your head

Nested JSON is a tree. Decide which branch becomes which table/record grain before flattening, otherwise arrays can multiply parent rows unexpectedly.

5. New words before we use them

JSON can represent nested objects and arrays, so flattening changes shape. Decide the target grain first. Preserve the raw payload, then extract only needed paths. An event array normally means one output row per event, not one row per whole document. Missing nested keys need an explicit policy.

6. Useful video / source

Source: DataTalksClub — Docker/PostgreSQL data-engineering workshop

Assigned scope: Use matching data-ingestion / script / database chapter

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=IP8xXebHmuE)

URL: <https://www.youtube.com/watch?v=IP8xXebHmuE>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Flatten normal/input/tracking_events.json to one row per tracking event with event_id, shipment_id, region, scan_type, event_time and device_id. Keep the original JSON untouched. Open the matching Guided Labs section. Reproduce the pattern once with the normal ParcelFlow fixtures while saving commands/output rather than transcribing the lesson.

8. Expected result / what you should see

You should be able to parse nested JSON without silently dropping arrays/objects, flatten only the fields required by the target grain and preserve raw payload evidence and show a concrete result/state that another person can inspect.

Observable target: For JSON and nested-record ingestion: Run the independent input from a clean state, record the actual output/count/schema/hash or exception, and compare it with an independently stated expected control.

9. Why it worked

This works because parse nested JSON without silently dropping arrays/objects, flatten only the fields required by the target grain and preserve raw payload evidence is made explicit before the result is accepted. Your independent check is: Do not validate JSON and nested-record ingestion by appearance alone. The proof is complete, replayable receipt/state evidence with no silent loss or duplication.

10. Common beginner mistakes

1. Calling JSON and nested-record ingestion complete because the command or tool finished. 2. Skipping the independent control: Do not validate JSON and nested-record ingestion by appearance alone. 3. Changing several things after a failure instead of isolating the first broken step. 4. Overwriting the failed attempt instead of saving it as repair evidence.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Nested JSON is a tree. 3. Run the earliest independent control: Do not validate JSON and nested-record ingestion by appearance alone. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Flatten a changed nested JSON payload whose root contains devices[] and each device contains readings[]. State output grain before code.

13. Validate your result

Do not validate JSON and nested-record ingestion by appearance alone. Use the lesson-specific control: Save the source/input identity, intended row/event grain, counts before/after, explicit error/quarantine evidence where relevant, and one independent reconciliation. Record the actual observed value/state and compare it with what the task requires. If they differ, the lesson is not yet mastered.

14. Teach it back in your own words

Without notes, explain JSON and nested-record ingestion in your own words. Start with the purpose: Parse nested JSON without silently dropping arrays/objects, flatten only the fields required by the target grain and preserve raw payload evidence. Then name one realistic failure mode and explain how you would prove the result using this check: Do not validate JSON and nested-record ingestion by appearance alone.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for JSON and nested-record ingestion with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE08-04.

DE08-04 - Parquet ingestion and metadata inspection

1. Why it matters

Explain columnar Parquet layout, verify file identity/magic/footer metadata, and inspect schema/row-group metadata with a compatible reader when available. This matters because ingestion is where outside uncertainty enters the system; weak boundaries can lose, duplicate or reshape data before anyone notices.

2. What you are learning

Explain columnar Parquet layout, verify file identity/magic/footer metadata, and inspect schema/row-group metadata with a compatible reader when available.

3. Prerequisite / what you already know

DE08-03 mastered.

4. Picture it in your head

Parquet is a columnar package with embedded schema/metadata. Inspect metadata first, then read only the required columns/partitions when possible.

5. New words before we use them

Parquet is a column-oriented binary format. A file contains column chunks organized in row groups plus footer metadata; readers use metadata to locate needed columns. The format begins and ends with PAR1 magic. Metadata inspection is not the same as opening a text file. A compatible Parquet engine such as PyArrow exposes schema and row-group details.

6. Useful video / source

Source: DataTalksClub DE Zoomcamp 2026 — From APIs to Warehouses with dlt

Assigned scope: Long-form main class — file/API ingestion and warehouse loading; use Parquet/file portions when reached

How to use it: Long or short video is allowed when it teaches the competency well. Watch actively without handwritten notes; type/run/follow when relevant. Use the Teaching Book as ready-made notes. Close video/review/Al solving before the independent task.

OPEN VIDEO / SOURCE - CLICK HERE (youtube.com)

URL: <https://www.youtube.com/watch?v=5eMytPBgmVs>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Run `m08_parquet_probe.py` on the supplied real Apache Parquet fixture. Prove magic at both ends and footer metadata length. If PyArrow is installed, also record row count, row groups and schema; otherwise record the dependency honestly. Open the matching Guided Labs section. Reproduce the pattern once with the normal ParcelFlow fixtures while saving commands/output rather than transcribing the lesson.

8. Expected result / what you should see

You should be able to explain columnar Parquet layout, verify file identity/magic/footer metadata, and inspect schema/row-group metadata with a compatible reader when available and show a concrete result/state that another person can inspect. Observable target: For Parquet ingestion and metadata inspection: Inspect schema/metadata, row count and one known aggregate; for pruning/partitioning, verify the reader can skip irrelevant columns/partitions rather than inferring it from folder names.

9. Why it worked

This works because explain columnar Parquet layout, verify file identity/magic/footer metadata, and inspect schema/row-group metadata with a compatible reader when available is made explicit before the result is accepted. Your independent check is: Do not validate Parquet ingestion and metadata inspection by appearance alone. The proof is complete, replayable receipt/state evidence with no silent loss or duplication.

10. Common beginner mistakes

1. Calling Parquet ingestion and metadata inspection complete because the command or tool finished. 2. Skipping the independent control: Do not validate Parquet ingestion and metadata inspection by appearance alone. 3. Changing several things after a failure instead of isolating the first broken step. 4. Overwriting the failed attempt instead of saving it as repair evidence.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Parquet is a columnar package with embedded schema/metadata. 3. Run the earliest independent control: Do not validate Parquet ingestion and metadata inspection by appearance alone. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Inspect the supplied Parquet file. Without a reader, prove binary identity/footer safely; with PyArrow, record schema/row groups/rows. Explain what you could not verify if PyArrow is unavailable.

13. Validate your result

Do not validate Parquet ingestion and metadata inspection by appearance alone. Use the lesson-specific control: Save the source/input identity, intended row/event grain, counts before/after, explicit error/quarantine evidence where relevant, and one independent reconciliation. Record the actual observed value/state and compare it with what the task requires. If they differ, the lesson is not yet mastered.

14. Teach it back in your own words

Without notes, explain Parquet ingestion and metadata inspection in your own words. Start with the purpose: Explain columnar Parquet layout, verify file identity/magic/footer metadata, and inspect schema/row-group metadata with a compatible reader when available. Then name one realistic failure mode and explain how you would prove the result using this check: Do not validate Parquet ingestion and metadata inspection by appearance alone.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Parquet ingestion and metadata inspection with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE08-05.

DE08-05 - REST API contracts, status codes and timeouts

1. Why it matters

Call an HTTP API with explicit timeout/status handling and prove that JSON decoding alone is not success evidence. This matters because ingestion is where outside uncertainty enters the system; weak boundaries can lose, duplicate or reshape data before anyone notices.

2. What you are learning

Call an HTTP API with explicit timeout/status handling and prove that JSON decoding alone is not success evidence.

3. Prerequisite / what you already know

DE08-04 mastered.

4. Picture it in your head

Git has three useful areas: working tree -> staging area -> commit history. status tells where changes are; add selects the snapshot; commit records it.

5. New words before we use them

HTTP success is a contract, not "I got JSON". A server can return JSON error details with a 4xx/5xx status. Set a timeout, check status before trusting payload, separate connection/timeouts from HTTP errors, and keep enough request metadata to reproduce a failure without logging secrets.

6. Useful video / source

Source: DataTalksClub — Docker/PostgreSQL data-engineering workshop

Assigned scope: Use matching data-ingestion / script / database chapter

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=IP8xXebHmuE)

URL: <https://www.youtube.com/watch?v=IP8xXebHmuE>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Call the localhost /health and /records endpoints. Prove /records without auth is 401. Call /slow with a short timeout and save the timeout evidence. Open the matching Guided Labs section. Reproduce the pattern once with the normal ParcelFlow fixtures while saving commands/output rather than transcribing the lesson.

8. Expected result / what you should see

You should be able to call an HTTP API with explicit timeout/status handling and prove that JSON decoding alone is not success evidence and show a concrete result/state that another person can inspect. Observable target: For REST API contracts, status codes and timeouts: Save status code, response/page count, record count and final pagination/checkpoint state; retry a controlled failure and prove no page is lost or duplicated.

9. Why it worked

The result is trustworthy because structure is treated as a contract rather than inferred casually; the validation checks fields/types/constraints against the declared shape. Your independent check is: Do not validate REST API contracts, status codes and timeouts by appearance alone. The proof is complete, replayable receipt/state evidence with no silent loss or duplication.

10. Common beginner mistakes

1. Calling REST API contracts, status codes and timeouts complete because the command or tool finished. 2. Skipping the independent control: Do not validate REST API contracts, status codes and timeouts by appearance alone. 3. Changing several things after a failure instead of isolating the first broken step. 4. Overwriting the failed attempt instead of saving it as repair evidence.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Git has three useful areas: working tree -> staging area -> commit history. 3. Run the earliest independent control: Do not validate REST API contracts, status codes and timeouts by appearance alone. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Write a function that accepts URL + timeout, rejects non-2xx responses before trusting JSON, and distinguishes HTTP error from timeout/connection error.

13. Validate your result

Do not validate REST API contracts, status codes and timeouts by appearance alone. Use the lesson-specific control: Save the source/input identity, intended row/event grain, counts before/after, explicit error/quarantine evidence where relevant, and one independent reconciliation. Record the actual observed value/state and compare it with what the task requires. If they differ, the lesson is not yet mastered.

14. Teach it back in your own words

Without notes, explain REST API contracts, status codes and timeouts in your own words. Start with the purpose: Call an HTTP API with explicit timeout/status handling and prove that JSON decoding alone is not success evidence. Then name one realistic failure mode and explain how you would prove the result using this check: Do not validate REST API contracts, status codes and timeouts by appearance alone.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for REST API contracts, status codes and timeouts with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE08-06.

DE08-06 - API authentication, pagination and rate limits

1. Why it matters

Keep credentials out of code, traverse pagination exactly once, obey rate-limit signals and reconcile fetched record counts. This matters because ingestion is where outside uncertainty enters the system; weak boundaries can lose, duplicate or reshape data before anyone notices.

2. What you are learning

Keep credentials out of code, traverse pagination exactly once, obey rate-limit signals and reconcile fetched record counts.

3. Prerequisite / what you already know

DE08-05 mastered.

4. Picture it in your head

Pagination is turning through numbered pages until the server says there are no more; authentication is showing a credential at the door. The secret belongs in configuration, not source code/logs.

5. New words before we use them

Credentials belong in environment/configuration boundaries, not committed source. Pagination means one logical dataset spans several responses. Follow the API next-page contract until it ends, protect against loops, and reconcile fetched IDs/counts. Rate limits are part of the source contract; respect Retry-After or documented quota behavior.

6. Useful video / source

Source: DataTalksClub DE Zoomcamp 2026 — From APIs to Warehouses with dlt

Assigned scope: Long-form main class — API extraction, authentication/pagination/rate-limit reliability concepts

How to use it: Long or short video is allowed when it teaches the competency well. Watch actively without handwritten notes; type/run/follow when relevant. Use the Teaching Book as ready-made notes. Close video/review/AI solving before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=5eMytPBgmVs)

URL: <https://www.youtube.com/watch?v=5eMytPBgmVs>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Fetch all localhost records with Bearer token m08-demo-token. The data spans three logical pages. Save IDs and prove no duplicate page/record traversal. Open the matching Guided Labs section. Reproduce the pattern once with the normal ParcelFlow fixtures while saving commands/output rather than transcribing the lesson.

8. Expected result / what you should see

You should be able to keep credentials out of code, traverse pagination exactly once, obey rate-limit signals and reconcile fetched record counts and show a concrete result/state that another person can inspect. Observable target: For API authentication, pagination and rate limits: Save status code, response/page count, record count and final pagination/checkpoint state; retry a controlled failure and prove no page is lost or duplicated.

9. Why it worked

The ingestion is trustworthy because request boundaries, response status/schema and progress are checked explicitly; arrival of a response is not treated as proof that the data is complete or valid. Your independent check is: Do not validate API authentication, pagination and rate limits by appearance alone. The proof is complete, replayable receipt/state evidence with no silent loss or duplication.

10. Common beginner mistakes

1. Calling API authentication, pagination and rate limits complete because the command or tool finished. 2. Skipping the independent control: Do not validate API authentication, pagination and rate limits by appearance alone. 3. Changing several things after a failure instead of isolating the first broken step. 4. Overwriting the failed attempt instead of saving it as repair evidence.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Pagination is turning through numbered pages until the server says there are no more; authentication is showing a credential at the door. 3. Run the earliest independent control: Do not validate API authentication, pagination and rate limits by appearance alone. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Build a paginator that stops only when next_page is null, refuses a repeated page token, keeps token outside source code and reconciles unique IDs.

13. Validate your result

Do not validate API authentication, pagination and rate limits by appearance alone. Use the lesson-specific control: Save the source/input identity, intended row/event grain, counts before/after, explicit error/quarantine evidence where relevant, and one independent reconciliation. Record the actual observed value/state and compare it with what the task requires. If they differ, the lesson is not yet mastered.

14. Teach it back in your own words

Without notes, explain API authentication, pagination and rate limits in your own words. Start with the purpose: Keep credentials out of code, traverse pagination exactly once, obey rate-limit signals and reconcile fetched record counts. Then name one realistic failure mode and explain how you would prove the result using this check: Do not validate API authentication, pagination and rate limits by appearance alone.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for API authentication, pagination and rate limits with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE08-07.

DE08-07 - Retries, backoff, jitter and transient failures

1. Why it matters

Retry only appropriate transient/idempotent operations with bounded attempts, exponential backoff/jitter and observable retry evidence. This matters because ingestion is where outside uncertainty enters the system; weak boundaries can lose, duplicate or reshape data before anyone notices.

2. What you are learning

Retry only appropriate transient/idempotent operations with bounded attempts, exponential backoff/jitter and observable retry evidence.

3. Prerequisite / what you already know

DE08-06 mastered.

4. Picture it in your head

Backoff increases delay between retries; jitter adds randomness so many workers do not retry together. Retry only failures that are plausibly transient.

5. New words before we use them

Retries help only when failure may be transient. Bound attempts. Retry appropriate idempotent requests/statuses, not every error. Exponential backoff reduces repeated pressure; jitter prevents many clients from retrying in lockstep. Every retry should be observable so hidden instability is not mistaken for a clean run.

6. Useful video / source

Source: DataTalksClub DE Zoomcamp 2026 — From APIs to Warehouses with dlt

Assigned scope: Long-form main class — reliable ingestion/failure handling; course lab adds explicit backoff/jitter evidence

How to use it: Long or short video is allowed when it teaches the competency well. Watch actively without handwritten notes; type/run/follow when relevant. Use the Teaching Book as ready-made notes. Close video/review/AI solving before the independent task.

OPEN VIDEO / SOURCE - CLICK HERE ([youtube.com](https://www.youtube.com/watch?v=5eMytPBgmVs))

URL: <https://www.youtube.com/watch?v=5eMytPBgmVs>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Use the guided client against the localhost server. Page 2 rate-limits once and page 3 returns one temporary 503. Prove the client eventually retrieves all records and logs exactly which attempts were retried. Open the matching Guided Labs section. Reproduce the pattern once with the normal ParcelFlow fixtures while saving commands/output rather than transcribing the lesson.

8. Expected result / what you should see

You should be able to retry only appropriate transient/idempotent operations with bounded attempts, exponential backoff/jitter and observable retry evidence and show a concrete result/state that another person can inspect. Observable target: For Retries, backoff, jitter and transient failures: record the actual before/after state and verify the lesson requirement with an independent control.

9. Why it worked

This works because retry only appropriate transient/idempotent operations with bounded attempts, exponential backoff/jitter and observable retry evidence is made explicit before the result is accepted. Your independent check is: Do not validate Retries, backoff, jitter and transient failures by appearance alone. The proof is complete, replayable receipt/state evidence with no silent loss or duplication.

10. Common beginner mistakes

1. Calling Retries, backoff, jitter and transient failures complete because the command or tool finished. 2. Skipping the independent control: Do not validate Retries, backoff, jitter and transient failures by appearance alone. 3. Changing several things after a failure instead of isolating the first broken step. 4. Overwriting the failed attempt instead of saving it as repair evidence.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Backoff increases delay between retries; jitter adds randomness so many workers do not retry together. 3. Run the earliest independent control: Do not validate Retries, backoff, jitter and transient failures by appearance alone. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Implement bounded retry for GET on 429/503 only. Record attempt number/status/backoff. Explain why blindly retrying POST can be unsafe.

13. Validate your result

Do not validate Retries, backoff, jitter and transient failures by appearance alone. Use the lesson-specific control: Save the source/input identity, intended row/event grain, counts before/after, explicit error/quarantine evidence where relevant, and one independent reconciliation. Record the actual observed value/state and compare it with what the task requires. If they differ, the lesson is not yet mastered.

14. Teach it back in your own words

Without notes, explain Retries, backoff, jitter and transient failures in your own words. Start with the purpose: Retry only appropriate transient/idempotent operations with bounded attempts, exponential backoff/jitter and observable retry evidence. Then name one realistic failure mode and explain how you would prove the result using this check: Do not validate Retries, backoff, jitter and transient failures by appearance alone.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Retries, backoff, jitter and transient failures with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE08-08.

DE08-08 - Database extraction and safe query boundaries

1. Why it matters

Extract from a database with parameterized read queries, explicit columns, deterministic bounds and no accidental source mutation. This matters because ingestion is where outside uncertainty enters the system; weak boundaries can lose, duplicate or reshape data before anyone notices.

2. What you are learning

Extract from a database with parameterized read queries, explicit columns, deterministic bounds and no accidental source mutation.

3. Prerequisite / what you already know

DE08-07 mastered; M07 database modeling complete.

4. Picture it in your head

Imagine a ZIP as a sealed travel bag. Extracting opens a working copy; copying keeps the original in place; moving changes location; renaming changes the label, not the contents.

5. New words before we use them

Database extraction should be boring and safe: explicit columns, read-only SELECT logic, parameters instead of string-built values, deterministic ordering/bounds, and a known extraction grain. Large production extracts often require chunking/snapshots/replica strategies, but the learner must first prove boundary correctness.

6. Useful video / source

Source: DataTalksClub — Docker/PostgreSQL data-engineering workshop

Assigned scope: Use matching data-ingestion / script / database chapter

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=IP8xXebHmuE)

URL: <https://www.youtube.com/watch?v=IP8xXebHmuE>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Query source_extract.db for orders updated from 10:00 inclusive to 12:30 exclusive using parameter placeholders. Save row count and SQL text. Do not UPDATE/DELETE the source. Open the matching Guided Labs section. Reproduce the pattern once with the normal ParcelFlow fixtures while saving commands/output rather than transcribing the lesson.

8. Expected result / what you should see

You should be able to extract from a database with parameterized read queries, explicit columns, deterministic bounds and no accidental source mutation and show a concrete result/state that another person can inspect. Observable target: For Database extraction and safe query boundaries: Query the database/schema context, inspect the created object/constraint, and run one positive plus one deliberately invalid case to prove the rule is actually enforced.

9. Why it worked

This works because extract from a database with parameterized read queries, explicit columns, deterministic bounds and no accidental source mutation is made explicit before the result is accepted. Your independent check is: Do not validate Database extraction and safe query boundaries by appearance alone. The proof is complete, replayable receipt/state evidence with no silent loss or duplication.

10. Common beginner mistakes

1. Calling Database extraction and safe query boundaries complete because the command or tool finished. 2. Skipping the independent control: Do not validate Database extraction and safe query boundaries by appearance alone. 3. Changing several things after a failure instead of isolating the first broken step. 4. Overwriting the failed attempt instead of saving it as repair evidence.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Imagine a ZIP as a sealed travel bag. 3. Run the earliest independent control: Do not validate Database extraction and safe query boundaries by appearance alone. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Parameterize a time-window SELECT against the SQLite source. Return explicit columns and deterministic order; save query + parameters + count.

13. Validate your result

Do not validate Database extraction and safe query boundaries by appearance alone. Use the lesson-specific control: Save the source/input identity, intended row/event grain, counts before/after, explicit error/quarantine evidence where relevant, and one independent reconciliation. Record the actual observed value/state and compare it with what the task requires. If they differ, the lesson is not yet mastered.

14. Teach it back in your own words

Without notes, explain Database extraction and safe query boundaries in your own words. Start with the purpose: Extract from a database with parameterized read queries, explicit columns, deterministic bounds and no accidental source mutation. Then name one realistic failure mode and explain how you would prove the result using this check: Do not validate Database extraction and safe query boundaries by appearance alone.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Database extraction and safe query boundaries with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE08-09.

DE08-09 - Historical batch landing and raw metadata

1. Why it matters

Land historical files without overwriting raw bytes; capture source name, size, hash, ingest time and batch/business date metadata. This matters because ingestion is where outside uncertainty enters the system; weak boundaries can lose, duplicate or reshape data before anyone notices.

2. What you are learning

Land historical files without overwriting raw bytes; capture source name, size, hash, ingest time and batch/business date metadata.

3. Prerequisite / what you already know

DE08-08 mastered.

4. Picture it in your head

Picture three zones: source outside your control, landing as an immutable receipt area, and staging as the workspace where parsing/validation begins. Do not blur those responsibilities.

5. New words before we use them

Historical backfills are easiest to trust when landing preserves each source object exactly once and metadata explains identity. Useful metadata includes original name/path, size, content hash, ingest time, business/batch date and source system. Landing is evidence, not a convenient scratch folder.

6. Useful video / source

Source: DataTalksClub — Docker/PostgreSQL data-engineering workshop

Assigned scope: Use matching data-ingestion / script / database chapter

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

OPEN VIDEO / SOURCE - CLICK HERE ([youtube.com](https://www.youtube.com/watch?v=IP8xXebHmuE))

URL: <https://www.youtube.com/watch?v=IP8xXebHmuE>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Land the three batch files to a clean landing directory. Record SHA-256 and business-row count for each. Rerun landing and prove existing bytes are unchanged. Open the matching Guided Labs section. Reproduce the pattern once with the normal ParcelFlow fixtures while saving commands/output rather than transcribing the lesson.

8. Expected result / what you should see

You should be able to land historical files without overwriting raw bytes; capture source name, size, hash, ingest time and batch/business date metadata and show a concrete result/state that another person can inspect. Observable target: For Historical batch landing and raw metadata: record the actual before/after state and verify the lesson requirement with an independent control.

9. Why it worked

The workflow is trustworthy because the received input is identified and preserved before transformation, so every downstream result can be traced back to unchanged source evidence. Your independent check is: Do not validate Historical batch landing and raw metadata by appearance alone. The proof is complete, replayable receipt/state evidence with no silent loss or duplication.

10. Common beginner mistakes

1. Calling Historical batch landing and raw metadata complete because the command or tool finished. 2. Skipping the independent control: Do not validate Historical batch landing and raw metadata by appearance alone. 3. Changing several things after a failure instead of isolating the first broken step. 4. Overwriting the failed attempt instead of saving it as repair evidence.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Picture three zones: source outside your control, landing as an immutable receipt area, and staging as the workspace where parsing/validation begins. 3. Run the earliest independent control: Do not validate Historical batch landing and raw metadata by appearance alone. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Land three changed batch files under a run/date prefix. Save name, bytes, SHA-256, business rows and ingest timestamp; rerun without overwriting changed bytes.

13. Validate your result

Do not validate Historical batch landing and raw metadata by appearance alone. Use the lesson-specific control: Save the source/input identity, intended row/event grain, counts before/after, explicit error/quarantine evidence where relevant, and one independent reconciliation. Record the actual observed value/state and compare it with what the task requires. If they differ, the lesson is not yet mastered.

14. Teach it back in your own words

Without notes, explain Historical batch landing and raw metadata in your own words. Start with the purpose: Land historical files without overwriting raw bytes; capture source name, size, hash, ingest time and batch/business date metadata. Then name one realistic failure mode and explain how you would prove the result using this check: Do not validate Historical batch landing and raw metadata by appearance alone.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Historical batch landing and raw metadata with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE08-10.

DE08-10 - Incremental loads and high-water marks

1. Why it matters

Load only new/changed rows using a stable cursor/high-water rule that does not lose ties at the boundary. This matters because ingestion is where outside uncertainty enters the system; weak boundaries can lose, duplicate or reshape data before anyone notices.

2. What you are learning

Load only new/changed rows using a stable cursor/high-water rule that does not lose ties at the boundary.

3. Prerequisite / what you already know

DE08-09 mastered.

4. Picture it in your head

A high-water mark is a bookmark in ordered source change. Read only records after the saved bookmark, but advance the bookmark only after the batch is safely committed.

5. New words before we use them

A high-water mark tells the next run where to continue. Timestamp-only cursors can lose rows when several records share the same timestamp. A composite cursor such as (updated_at, record_id) gives a deterministic total order. State should move only after the destination commit succeeds.

6. Useful video / source

Source: DataTalksClub — Docker/PostgreSQL data-engineering workshop

Assigned scope: Use matching data-ingestion / script / database chapter

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=IP8xXebHmuE)

URL: <https://www.youtube.com/watch?v=IP8xXebHmuE>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Process incremental_source.csv in two runs: first five rows, then the remainder. Use (updated_at, record_id) as cursor. Prove I006 is not lost even though I004-I006 share 10:10. Open the matching Guided Labs section. Reproduce the pattern once with the normal ParcelFlow fixtures while saving commands/output rather than transcribing the lesson.

8. Expected result / what you should see

You should be able to load only new/changed rows using a stable cursor/high-water rule that does not lose ties at the boundary and show a concrete result/state that another person can inspect. Observable target: For Incremental loads and high-water marks: record the actual before/after state and verify the lesson requirement with an independent control.

9. Why it worked

This works because load only new/changed rows using a stable cursor/high-water rule that does not lose ties at the boundary is made explicit before the result is accepted. Your independent check is: Do not validate Incremental loads and high-water marks by appearance alone. The proof is complete, replayable receipt/state evidence with no silent loss or duplication.

10. Common beginner mistakes

1. Calling Incremental loads and high-water marks complete because the command or tool finished. 2. Skipping the independent control: Do not validate Incremental loads and high-water marks by appearance alone. 3. Changing several things after a failure instead of isolating the first broken step. 4. Overwriting the failed attempt instead of saving it as repair evidence.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: A high-water mark is a bookmark in ordered source change. 3. Run the earliest independent control: Do not validate Incremental loads and high-water marks by appearance alone. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Process the 8-row incremental fixture using composite cursor. Stop first run after five rows, then finish; prove tie-boundary row I006 is not lost and third run returns zero.

13. Validate your result

Do not validate Incremental loads and high-water marks by appearance alone. Use the lesson-specific control: Save the source/input identity, intended row/event grain, counts before/after, explicit error/quarantine evidence where relevant, and one independent reconciliation. Record the actual observed value/state and compare it with what the task requires. If they differ, the lesson is not yet mastered.

14. Teach it back in your own words

Without notes, explain Incremental loads and high-water marks in your own words. Start with the purpose: Load only new/changed rows using a stable cursor/high-water rule that does not lose ties at the boundary. Then name one realistic failure mode and explain how you would prove the result using this check: Do not validate Incremental loads and high-water marks by appearance alone.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Incremental loads and high-water marks with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE08-11.

DE08-11 - CDC concepts and change-event semantics

1. Why it matters

Interpret insert/update/delete change events, before/after images and source ordering metadata without pretending a batch poll is CDC. This matters because ingestion is where outside uncertainty enters the system; weak boundaries can lose, duplicate or reshape data before anyone notices.

2. What you are learning

Interpret insert/update/delete change events, before/after images and source ordering metadata without pretending a batch poll is CDC.

3. Prerequisite / what you already know

DE08-10 mastered.

4. Picture it in your head

CDC is a stream of change facts - insert/update/delete - not merely current rows. Consumers must interpret operation type and ordering to reconstruct state correctly.

5. New words before we use them

CDC captures changes from a transaction/change log and emits operation semantics such as create/update/delete, often with before/after state and source ordering metadata. Polling a table by updated_at is incremental extraction, not true CDC. Downstream consumers must define delete handling, ordering and replay behavior.

6. Useful video / source

Source: Binod Suman Academy — Change Data Capture with Debezium/Kafka

Assigned scope: What/why/how CDC + Debezium/MySQL/Kafka demo

How to use it: Use for the CDC event mental model and real demo. Teaching Book remains authoritative for generic change-event semantics and failure reasoning.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=dN_11nBcv_A)

URL: https://www.youtube.com/watch?v=dN_11nBcv_A

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Apply cdc_events.jsonl in seq order to an in-memory current-state map. Prove A1 is updated, A2 is deleted and A3 exists. Explain why the event log remains valuable even after current state is reconstructed. Open the matching Guided Labs section. Reproduce the pattern once with the normal ParcelFlow fixtures while saving commands/output rather than transcribing the lesson.

8. Expected result / what you should see

You should be able to interpret insert/update/delete change events, before/after images and source ordering metadata without pretending a batch poll is CDC and show a concrete result/state that another person can inspect. Observable target: For CDC concepts and change-event semantics: record the actual before/after state and verify the lesson requirement with an independent control.

9. Why it worked

This works because interpret insert/update/delete change events, before/after images and source ordering metadata without pretending a batch poll is CDC is made explicit before the result is accepted. Your independent check is: Do not validate CDC concepts and change-event semantics by appearance alone. The proof is complete, replayable receipt/state evidence with no silent loss or duplication.

10. Common beginner mistakes

1. Calling CDC concepts and change-event semantics complete because the command or tool finished. 2. Skipping the independent control: Do not validate CDC concepts and change-event semantics by appearance alone. 3. Changing several things after a failure instead of isolating the first broken step. 4. Overwriting the failed attempt instead of saving it as repair evidence.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: CDC is a stream of change facts - insert/update/delete - not merely current rows. 3. Run the earliest independent control: Do not validate CDC concepts and change-event semantics by appearance alone. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Consume the supplied CDC log to derive current state plus an audit trail. State delete policy and what sequence field protects.

13. Validate your result

Do not validate CDC concepts and change-event semantics by appearance alone. Use the lesson-specific control: Save the source/input identity, intended row/event grain, counts before/after, explicit error/quarantine evidence where relevant, and one independent reconciliation. Record the actual observed value/state and compare it with what the task requires. If they differ, the lesson is not yet mastered.

14. Teach it back in your own words

Without notes, explain CDC concepts and change-event semantics in your own words. Start with the purpose: Interpret insert/update/delete change events, before/after images and source ordering metadata without pretending a batch poll is CDC. Then name one realistic failure mode and explain how you would prove the result using this check: Do not validate CDC concepts and change-event semantics by appearance alone.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for CDC concepts and change-event semantics with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE08-12.

DE08-12 - Schema drift, compatibility, time zones and quarantine

1. Why it matters

Detect compatible/incompatible drift, normalize timestamps to UTC, quarantine invalid rows and avoid silently coercing bad values. This matters because ingestion is where outside uncertainty enters the system; weak boundaries can lose, duplicate or reshape data before anyone notices.

2. What you are learning

Detect compatible/incompatible drift, normalize timestamps to UTC, quarantine invalid rows and avoid silently coercing bad values.

3. Prerequisite / what you already know

DE08-11 mastered.

4. Picture it in your head

Schema drift is the input contract changing under you. Compatible changes can be handled deliberately; incompatible rows belong in quarantine with the exact reason and original payload.

5. New words before we use them

Schema drift is any change to expected fields/types/meaning. Some changes are additive/compatible; others break consumers. Timestamp strings also carry semantics: normalize instants to UTC while preserving enough raw evidence to diagnose source issues. Invalid rows go to quarantine with reason; do not replace bad numeric/time values with zero.

6. Useful video / source

Source: DataTalksClub DE Zoomcamp 2026 — From APIs to Warehouses with dlt

Assigned scope: Long-form main class — schema normalization/change validation; book adds compatibility/time-zone/quarantine rules

How to use it: Long or short video is allowed when it teaches the competency well. Watch actively without handwritten notes; type/run/follow when relevant. Use the Teaching Book as ready-made notes. Close video/review/AI solving before the independent task.

OPEN VIDEO / SOURCE - CLICK HERE ([youtube.com](https://www.youtube.com/watch?v=5eMytPBgmVs))

URL: <https://www.youtube.com/watch?v=5eMytPBgmVs>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Process drift_events.csv. Parse amount + event_time; normalize valid times to UTC. Quarantine bad numeric and bad timestamp rows. Reconcile all six source rows. Open the matching Guided Labs section. Reproduce the pattern once with the normal ParcelFlow fixtures while saving commands/output rather than transcribing the lesson.

8. Expected result / what you should see

You should be able to detect compatible/incompatible drift, normalize timestamps to UTC, quarantine invalid rows and avoid silently coercing bad values and show a concrete result/state that another person can inspect. Observable target: For Schema drift, compatibility, time zones and quarantine: Query the database/schema context, inspect the created object/constraint, and run one positive plus one deliberately invalid case to prove the rule is actually enforced.

9. Why it worked

The result is trustworthy because structure is treated as a contract rather than inferred casually; the validation checks fields/types/constraints against the declared shape. Your independent check is: Do not validate Schema drift, compatibility, time zones and quarantine by appearance alone. The proof is complete, replayable receipt/state evidence with no silent loss or duplication.

10. Common beginner mistakes

1. Calling Schema drift, compatibility, time zones and quarantine complete because the command or tool finished. 2. Skipping the independent control: Do not validate Schema drift, compatibility, time zones and quarantine by appearance alone. 3. Changing several things after a failure instead of isolating the first broken step. 4. Overwriting the failed attempt instead of saving it as repair evidence.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Schema drift is the input contract changing under you. 3. Run the earliest independent control: Do not validate Schema drift, compatibility, time zones and quarantine by appearance alone. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Write a contract validator that accepts extra optional field `source_note`, but quarantines invalid amount/time. Normalize valid timestamps to UTC and preserve raw row.

13. Validate your result

Do not validate Schema drift, compatibility, time zones and quarantine by appearance alone. Use the lesson-specific control: Save the source/input identity, intended row/event grain, counts before/after, explicit error/quarantine evidence where relevant, and one independent reconciliation. Record the actual observed value/state and compare it with what the task requires. If they differ, the lesson is not yet mastered.

14. Teach it back in your own words

Without notes, explain Schema drift, compatibility, time zones and quarantine in your own words. Start with the purpose: Detect compatible/incompatible drift, normalize timestamps to UTC, quarantine invalid rows and avoid silently coercing bad values. Then name one realistic failure mode and explain how you would prove the result using this check: Do not validate Schema drift, compatibility, time zones and quarantine by appearance alone.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Schema drift, compatibility, time zones and quarantine with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE08-13.

DE08-13 - Checkpoint/state management and replay safety

1. Why it matters

Persist checkpoint state only after successful durable output and prove a crash/replay cannot skip uncommitted data. This matters because ingestion is where outside uncertainty enters the system; weak boundaries can lose, duplicate or reshape data before anyone notices.

2. What you are learning

Persist checkpoint state only after successful durable output and prove a crash/replay cannot skip uncommitted data.

3. Prerequisite / what you already know

DE08-12 mastered.

4. Picture it in your head

A checkpoint is a durable bookmark of progress. Replay safety means you can restart from a known checkpoint without losing data or creating duplicate side effects.

5. New words before we use them

Checkpoint state is part of correctness. If you advance a cursor before output is durably committed, a crash can make the next run skip data forever. Correct order is: read from old state -> write/validate -> commit output -> atomically persist new state. Replay from an old state must be safe.

6. Useful video / source

Source: DataTalksClub DE Zoomcamp 2026 — From APIs to Warehouses with dlt

Assigned scope: Long-form main class — incremental/stateful ingestion; course task proves checkpoint/replay safety

How to use it: Long or short video is allowed when it teaches the competency well. Watch actively without handwritten notes; type/run/follow when relevant. Use the Teaching Book as ready-made notes. Close video/review/AI solving before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=5eMytPBgmVs)

URL: <https://www.youtube.com/watch?v=5eMytPBgmVs>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Simulate destination failure after candidate I006 is read but before commit. Prove stored state remains I005; rerun should read I006 again rather than skip it. Open the matching Guided Labs section. Reproduce the pattern once with the normal ParcelFlow fixtures while saving commands/output rather than transcribing the lesson.

8. Expected result / what you should see

You should be able to persist checkpoint state only after successful durable output and prove a crash/replay cannot skip uncommitted data and show a concrete result/state that another person can inspect. Observable target: For Checkpoint/state management and replay safety: record the actual before/after state and verify the lesson requirement with an independent control.

9. Why it worked

The workflow is trustworthy because progress/state is explicit and replaying the same input is tested to converge without silent loss or duplicate side effects. Your independent check is: Do not validate Checkpoint/state management and replay safety by appearance alone. The proof is complete, replayable receipt/state evidence with no silent loss or duplication.

10. Common beginner mistakes

1. Calling Checkpoint/state management and replay safety complete because the command or tool finished. 2. Skipping the independent control: Do not validate Checkpoint/state management and replay safety by appearance alone. 3. Changing several things after a failure instead of isolating the first broken step. 4. Overwriting the failed attempt instead of saving it as repair evidence.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: A checkpoint is a durable bookmark of progress. 3. Run the earliest independent control: Do not validate Checkpoint/state management and replay safety by appearance alone. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Write pseudocode or Python showing state is saved only after destination success. Inject a failure before commit and prove state did not advance.

13. Validate your result

Do not validate Checkpoint/state management and replay safety by appearance alone. Use the lesson-specific control: Save the source/input identity, intended row/event grain, counts before/after, explicit error/quarantine evidence where relevant, and one independent reconciliation. Record the actual observed value/state and compare it with what the task requires. If they differ, the lesson is not yet mastered.

14. Teach it back in your own words

Without notes, explain Checkpoint/state management and replay safety in your own words. Start with the purpose: Persist checkpoint state only after successful durable output and prove a crash/replay cannot skip uncommitted data. Then name one realistic failure mode and explain how you would prove the result using this check: Do not validate Checkpoint/state management and replay safety by appearance alone.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Checkpoint/state management and replay safety with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE08-14.

DE08-14 - Idempotent landing and content identity

1. Why it matters

Use stable business/content identity so rerunning the same input does not create duplicate output while changed content remains detectable. This matters because ingestion is where outside uncertainty enters the system; weak boundaries can lose, duplicate or reshape data before anyone notices.

2. What you are learning

Use stable business/content identity so rerunning the same input does not create duplicate output while changed content remains detectable.

3. Prerequisite / what you already know

DE08-13 mastered.

4. Picture it in your head

Picture three zones: source outside your control, landing as an immutable receipt area, and staging as the workspace where parsing/validation begins. Do not blur those responsibilities.

5. New words before we use them

Idempotency means repeating the same operation produces the same final state. For files, SHA-256 can be a strong content identity signal; business/event keys may be the dedup identity for records. Do not confuse filename with content identity: a renamed duplicate file can still be identical bytes.

6. Useful video / source

Source: PyCon US 2025 — Building Resilient Data Pipelines: The Power of Idempotency

Assigned scope: Full ~25-minute talk — pitfalls → idempotent design → pytest strategy

How to use it: Watch the talk, then implement the course idempotent landing/content-identity task. Watching alone does not pass the lesson.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=kT8YWX4NE2o)

URL: <https://www.youtube.com/watch?v=kT8YWX4NE2o>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Hash all landed batch files, then run landing twice. Prove destination contains one copy per intended file/content and hashes stay unchanged. Open the matching Guided Labs section. Reproduce the pattern once with the normal ParcelFlow fixtures while saving commands/output rather than transcribing the lesson.

8. Expected result / what you should see

You should be able to use stable business/content identity so rerunning the same input does not create duplicate output while changed content remains detectable and show a concrete result/state that another person can inspect. Observable target: For Idempotent landing and content identity: record the actual before/after state and verify the lesson requirement with an independent control.

9. Why it worked

The workflow is trustworthy because the received input is identified and preserved before transformation, so every downstream result can be traced back to unchanged source evidence. Your independent check is: Do not validate Idempotent landing and content identity by appearance alone. The proof is complete, replayable receipt/state evidence with no silent loss or duplication.

10. Common beginner mistakes

1. Calling Idempotent landing and content identity complete because the command or tool finished. 2. Skipping the independent control: Do not validate Idempotent landing and content identity by appearance alone. 3. Changing several things after a failure instead of isolating the first broken step. 4. Overwriting the failed attempt instead of saving it as repair evidence.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Picture three zones: source outside your control, landing as an immutable receipt area, and staging as the workspace where parsing/validation begins. 3. Run the earliest independent control: Do not validate Idempotent landing and content identity by appearance alone. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Use content hash + event key to prevent duplicates. Test same filename/same bytes, renamed/same bytes and same filename/changed bytes; define each outcome.

13. Validate your result

Do not validate Idempotent landing and content identity by appearance alone. Use the lesson-specific control: Save the source/input identity, intended row/event grain, counts before/after, explicit error/quarantine evidence where relevant, and one independent reconciliation. Record the actual observed value/state and compare it with what the task requires. If they differ, the lesson is not yet mastered.

14. Teach it back in your own words

Without notes, explain Idempotent landing and content identity in your own words. Start with the purpose: Use stable business/content identity so rerunning the same input does not create duplicate output while changed content remains detectable. Then name one realistic failure mode and explain how you would prove the result using this check: Do not validate Idempotent landing and content identity by appearance alone.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Idempotent landing and content identity with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE08-15.

DE08-15 - Late-arriving data, event time and processing time

1. Why it matters

Distinguish event time from arrival/processing time, classify lateness against an explicit tolerance and explain watermark trade-offs. This matters because ingestion is where outside uncertainty enters the system; weak boundaries can lose, duplicate or reshape data before anyone notices.

2. What you are learning

Distinguish event time from arrival/processing time, classify lateness against an explicit tolerance and explain watermark trade-offs.

3. Prerequisite / what you already know

DE08-14 mastered.

4. Picture it in your head

Event time is when the business event happened; processing time is when your system saw it. Late data is a record arriving after the normal processing window, not necessarily bad data.

5. New words before we use them

Event time says when the source event happened; processing/arrival time says when the pipeline saw it. Late data arrives after the allowed delay. Watermarks are progress assumptions used by streaming systems; a longer allowance improves completeness but increases latency. M08 teaches the concept without requiring Flink runtime.

6. Useful video / source

Source: Confluent Developer — Event Time and Watermarks

Assigned scope: 12-minute video: event time vs processing time, out-of-order events, watermarks and late data

How to use it: Use the official video to see why event time differs from processing time; course task applies the concept without requiring Flink mastery.

OPEN VIDEO / SOURCE - CLICK HERE (developer.confluent.io)

URL: <https://developer.confluent.io/courses/apache-flink/timely-stream-processing/>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

For `late_events.json`, classify each event using 10-minute allowed lateness. Save IDs and delay seconds of late events. Explain why sorting only by `arrived_at` changes the business timeline. Open the matching Guided Labs section. Reproduce the pattern once with the normal `ParcelFlow` fixtures while saving commands/output rather than transcribing the lesson.

8. Expected result / what you should see

You should be able to distinguish event time from arrival/processing time, classify lateness against an explicit tolerance and explain watermark trade-offs and show a concrete result/state that another person can inspect. Observable target: For Late-arriving data, event time and processing time: record the actual before/after state and verify the lesson requirement with an independent control.

9. Why it worked

This works because distinguish event time from arrival/processing time, classify lateness against an explicit tolerance and explain watermark trade-offs is made explicit before the result is accepted. Your independent check is: Do not validate Late-arriving data, event time and processing time by appearance alone. The proof is complete, replayable receipt/state evidence with no silent loss or duplication.

10. Common beginner mistakes

1. Calling Late-arriving data, event time and processing time complete because the command or tool finished. 2. Skipping the independent control: Do not validate Late-arriving data, event time and processing time by appearance alone. 3. Changing several things after a failure instead of isolating the first broken step. 4. Overwriting the failed attempt instead of saving it as repair evidence.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Event time is when the business event happened; processing time is when your system saw it. 3. Run the earliest independent control: Do not validate Late-arriving data, event time and processing time by appearance alone. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Classify the six late-event fixtures at 10-minute tolerance. Recompute at 20 minutes and explain completeness-vs-latency trade-off.

13. Validate your result

Do not validate Late-arriving data, event time and processing time by appearance alone. Use the lesson-specific control: Save the source/input identity, intended row/event grain, counts before/after, explicit error/quarantine evidence where relevant, and one independent reconciliation. Record the actual observed value/state and compare it with what the task requires. If they differ, the lesson is not yet mastered.

14. Teach it back in your own words

Without notes, explain Late-arriving data, event time and processing time in your own words. Start with the purpose: Distinguish event time from arrival/processing time, classify lateness against an explicit tolerance and explain watermark trade-offs. Then name one realistic failure mode and explain how you would prove the result using this check: Do not validate Late-arriving data, event time and processing time by appearance alone.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Late-arriving data, event time and processing time with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE08-16.

DE08-16 - Ingestion observability: counts, latency and run metadata

1. Why it matters

Emit run-level evidence for rows seen/accepted/quarantined, retries, bytes, latency, checkpoint and status so failures can be diagnosed. This matters because ingestion is where outside uncertainty enters the system; weak boundaries can lose, duplicate or reshape data before anyone notices.

2. What you are learning

Emit run-level evidence for rows seen/accepted/quarantined, retries, bytes, latency, checkpoint and status so failures can be diagnosed.

3. Prerequisite / what you already know

DE08-15 mastered.

4. Picture it in your head

Observability is the pipeline dashboard: how many records arrived, how late, which run processed them, what failed, and how long each stage took.

5. New words before we use them

Observable ingestion emits evidence, not just "success". At minimum capture run_id/time, source, input rows/bytes, accepted/quarantine counts, retry count, elapsed/latency, checkpoint before/after, destination result and final status/error. Metrics must reconcile and should not leak credentials.

6. Useful video / source

Source: DataTalksClub DE Zoomcamp 2026 — From APIs to Warehouses with dlt

Assigned scope: Long-form main class — pipeline metadata/validation/dashboard concepts; course lab adds counts/latency/run metadata

How to use it: Long or short video is allowed when it teaches the competency well. Watch actively without handwritten notes; type/run/follow when relevant. Use the Teaching Book as ready-made notes. Close video/review/AI solving before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=5eMytPBgmVs)

URL: <https://www.youtube.com/watch?v=5eMytPBgmVs>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Create one run record from the normal smoke job. Verify seen = accepted + quarantine where applicable and retries are visible rather than hidden. Open the matching Guided Labs section. Reproduce the pattern once with the normal ParcelFlow fixtures while saving commands/output rather than transcribing the lesson.

8. Expected result / what you should see

You should be able to emit run-level evidence for rows seen/accepted/quarantined, retries, bytes, latency, checkpoint and status so failures can be diagnosed and show a concrete result/state that another person can inspect. Observable target: For Ingestion observability: counts, latency and run metadata: record the actual before/after state and verify the lesson requirement with an independent control.

9. Why it worked

This works because emit run-level evidence for rows seen/accepted/quarantined, retries, bytes, latency, checkpoint and status so failures can be diagnosed is made explicit before the result is accepted. Your independent check is: Do not validate Ingestion observability: counts, latency and run metadata by appearance alone. The proof is complete, replayable receipt/state evidence with no silent loss or duplication.

10. Common beginner mistakes

1. Calling Ingestion observability: counts, latency and run metadata complete because the command or tool finished. 2. Skipping the independent control: Do not validate Ingestion observability: counts, latency and run metadata by appearance alone. 3. Changing several things after a failure instead of isolating the first broken step. 4. Overwriting the failed attempt instead of saving it as repair evidence.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Observability is the pipeline dashboard: how many records arrived, how late, which run processed them, what failed, and how long each stage took. 3. Run the earliest independent control: Do not validate Ingestion observability: counts, latency and run metadata by appearance alone. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Emit a JSONL run record with counts, bytes, retries, elapsed_ms, cursor_before/after, status and quarantine path. Prove no token/password is logged.

13. Validate your result

Do not validate Ingestion observability: counts, latency and run metadata by appearance alone. Use the lesson-specific control: Save the source/input identity, intended row/event grain, counts before/after, explicit error/quarantine evidence where relevant, and one independent reconciliation. Record the actual observed value/state and compare it with what the task requires. If they differ, the lesson is not yet mastered.

14. Teach it back in your own words

Without notes, explain Ingestion observability: counts, latency and run metadata in your own words. Start with the purpose: Emit run-level evidence for rows seen/accepted/quarantined, retries, bytes, latency, checkpoint and status so failures can be diagnosed. Then name one realistic failure mode and explain how you would prove the result using this check: Do not validate Ingestion observability: counts, latency and run metadata by appearance alone.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Ingestion observability: counts, latency and run metadata with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE08-17.

DE08-17 - Ingestion integration, reconciliation and backfill proof

1. Why it matters

Run an end-to-end ingestion job, rerun it idempotently, backfill historical data and reconcile destination records to source/control totals. This matters because ingestion is where outside uncertainty enters the system; weak boundaries can lose, duplicate or reshape data before anyone notices.

2. What you are learning

Run an end-to-end ingestion job, rerun it idempotently, backfill historical data and reconcile destination records to source/control totals.

3. Prerequisite / what you already know

DE08-01 through DE08-16 mastered.

4. Picture it in your head

Reconciliation is like closing a cash register: independently compare row counts, keys, and important totals to the source before trusting a cleaned or summarized result.

5. New words before we use them

Integration proves the pieces work together: inventory inputs -> land raw -> parse/validate -> extract/fetch -> deduplicate -> commit -> checkpoint -> observe -> reconcile. Then rerun unchanged input and perform a historical backfill. A backfill is not allowed to corrupt current state or duplicate already committed business events.

6. Useful video / source

Source: DataTalksClub — Docker/PostgreSQL data-engineering workshop

Assigned scope: Use matching data-ingestion / script / database chapter

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=IP8xXebHmuE)

URL: <https://www.youtube.com/watch?v=IP8xXebHmuE>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Run `m08_smoke_runner.py` from a clean normal/output folder. Save the generated JSON evidence. Rerun it and confirm landing/idempotency controls remain stable. Then explain how you would add one older batch without resetting unrelated state. Open the matching Guided Labs section. Reproduce the pattern once with the normal ParcelFlow fixtures while saving commands/output rather than transcribing the lesson.

8. Expected result / what you should see

You should be able to run an end-to-end ingestion job, rerun it idempotently, backfill historical data and reconcile destination records to source/control totals and show a concrete result/state that another person can inspect. Observable target: For Ingestion integration, reconciliation and backfill proof: Inspect DAG/task-instance state and logs for the intended logical interval; prove dependencies, retries or backfill behavior on a changed run rather than by reading code only.

9. Why it worked

This works because run an end-to-end ingestion job, rerun it idempotently, backfill historical data and reconcile destination records to source/control totals is made explicit before the result is accepted. Your independent check is: Do not validate Ingestion integration, reconciliation and backfill proof by appearance alone. The proof is complete, replayable receipt/state evidence with no silent loss or duplication.

10. Common beginner mistakes

1. Calling Ingestion integration, reconciliation and backfill proof complete because the command or tool finished. 2. Skipping the independent control: Do not validate Ingestion integration, reconciliation and backfill proof by appearance alone. 3. Changing several things after a failure instead of isolating the first broken step. 4. Overwriting the failed attempt instead of saving it as repair evidence.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Reconciliation is like closing a cash register: independently compare row counts, keys, and important totals to the source before trusting a cleaned or summarized result. 3. Run the earliest independent control: Do not validate Ingestion integration, reconciliation and backfill proof by appearance alone. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

From a clean output folder run the full normal ingestion smoke, rerun unchanged inputs, then add one hypothetical older batch. Write a reconciliation table and backfill safety explanation.

13. Validate your result

Do not validate Ingestion integration, reconciliation and backfill proof by appearance alone. Use the lesson-specific control: Save the source/input identity, intended row/event grain, counts before/after, explicit error/quarantine evidence where relevant, and one independent reconciliation. Record the actual observed value/state and compare it with what the task requires. If they differ, the lesson is not yet mastered.

14. Teach it back in your own words

Without notes, explain Ingestion integration, reconciliation and backfill proof in your own words. Start with the purpose: Run an end-to-end ingestion job, rerun it idempotently, backfill historical data and reconcile destination records to source/control totals. Then name one realistic failure mode and explain how you would prove the result using this check: Do not validate Ingestion integration, reconciliation and backfill proof by appearance alone.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Ingestion integration, reconciliation and backfill proof with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: module practical / Gate route after the mastery checklist.